Corrections to the Reading Report (2026-05-12)

Vincent-Lamarre et al. (2016). "The Latent Structure of Dictionaries." *Topics in Cognitive Science* 8(3): 625 – 659. DOI: 10.1111/tops.12211

Summary

Your report was a strong first reading. The high-level story — Kernel, Core, MinSet, and the NP-hardness of grounding — is correct. There are a few technical errors that matter for implementation, listed below from most to least critical.

---

1. Kernel Algorithm — Direction is Reversed ⚠ Critical

What you wrote: "*Iteratively remove words with in-degree zero.*"

What the paper says: Iteratively remove words with **out-degree zero**.

Why this matters

The edges in the dictionary graph point as follows:

```
add_edge(u, v)  means  "u appears in the definition of v"
                       (u defines v, or: u is a defining word of v)
```

A word with **out-degree zero** never appears in any other word' s definition. It is a *consumer* of the network — it takes definitions but contributes nothing to others. Removing it cannot make any word undefinable.

A word with **in-degree zero** is never *defined by* anything else in the vocabulary. That is a completely different property (it is actually what makes a word a candidate for the MinSet, not the Kernel).

Concrete example

```
Graph:   fast ---> speed ---> moving ---> fast   (3-cycle)
                  fast ---> quickly              (tail)

Edges mean:
```

```
"fast"    appears in definition of "speed"    (fast → speed)
"speed"   appears in definition of "moving"   (speed → moving)
"moving"  appears in definition of "fast"     (moving → fast)
"fast"    appears in definition of "quickly"  (fast → quickly)

out-degree:  fast=2,  speed=1,  moving=1,  quickly=0
in-degree:   fast=1,  speed=1,  moving=1,  quickly=1
```

Correct algorithm (out-degree = 0): 1. Remove *quickly* (out-degree = 0) 2. No more zero-out-degree nodes → stop 3. **Kernel = {fast, speed, moving}** ✓

Buggy algorithm (in-degree = 0) — your version: 1. No node has in-degree = 0 → nothing is removed 2. **"Kernel" = {fast, speed, moving, quickly}** ☒

quickly survives in the buggy Kernel even though it is just a peripheral word that depends on *fast* to be defined but is never used to define anything else.

Reference

Blondin Massé et al. (2008), Algorithm 2 (Computing the grounding kernel):

```
U ← {}
repeat
    U ← U ∪ { v ∈ V | N☒(v) ⊆ U }   # N☒(v) = out-neighbours of v
until U unchanged
Kernel ← V \ U
```

`N☒(v) ⊆ U` means all out-neighbours of v are already in the removal set, i.e. v itself has effective out-degree zero within the remaining graph.

---

2. Core Definition — Not Always the Largest SCC

What you wrote: "*The Core is the giant Strongly Connected Component (SCC) at the center of the Kernel.*"

What the paper says: The Core is the **union of all Source SCCs** in the condensation of the Kernel, where a *Source* is an SCC with **in-degree zero in the condensation DAG** (no arcs coming in from outside).

In two of the four dictionaries analysed, the Core equals the largest SCC. In the other two, it is the largest SCC *plus a few small SCCs* — an artefact of preprocessing, but technically distinct from "the largest SCC."

The authors define it as Sources, not as max-SCC. For implementation, use:

```
C = nx.condensation(Kernel_subgraph)
sources = {n for n in C if C.in_degree(n) == 0}
Core = union of C.nodes[scc_id]["members"] for scc_id in sources
```

---

3. MinSet Computation — ILP, Not Greedy

What you wrote: "*For large dictionaries ⋯ Approximate —greedy (large dictionaries).*"

What the paper says: All four dictionaries were solved with an **Integer Linear Program (ILP)** implemented in CPLEX. For Webster (~248,000 words) it ran *for days* and returned an "almost optimal" solution. The paper does not use a greedy algorithm —that was your own addition.

The greedy heuristic (pick the word in the most remaining cycles, remove it, repeat) is a reasonable approximation, but it is not what Vincent-Lamarre et al. did. If you implement it as a fallback for large graphs, label it clearly as `approximate=True`.

---

4. Preprocessing — Stemming, Not Lemmatisation

What you wrote: "*Lemmatization: Reduce every word to its base form.*"

What the paper says: The paper uses the word **"stemmatized"** (stemming), not lemmatisation. These are different operations:

Operation	Example	Tool
Stemming	"running" → "run" (may not be a real word)	Porter stemmer
Lemmatisation	"running" → "run" (always a valid lemma + POS)	WordNetLemmatizer

For our implementation we will use **lemmatisation** because it produces cleaner vocabulary matches with WordNet. This is a conscious deviation from the original; we will note it in the paper' s methodology section.

Also: the original paper processes **the first sense per part of speech**, not just the first sense of the entire word. "bank (n.)" and "bank (v.)" are treated as separate entries, each with its own first-sense definition.

---

5. Publication Year and Dictionaries Used

What you wrote: "*Vincent-Lamarre et al. (2014)***"**

Correct citation: > Vincent-Lamarre, P., Blondin Massé, A., Lopes, M., Lord, M., Marcotte, O., & Harnad, S. > (2016). The latent structure of dictionaries. > *Topics in Cognitive Science*, **8**(3), 625 – 659. https://doi.org/10.1111/tops.12211

2014 is the arXiv preprint date (v1: 2014-11-01). The peer-reviewed version is 2016. Always cite the journal version.

Dictionaries used: Longman, Cambridge, Merriam-Webster, WordNet. OALD and WordSmyth do **not** appear in the paper.

---

6. Tarjan vs. Kosaraju — Your Addition, Not the Paper

Your comparison of Tarjan and Kosaraju is technically accurate (Tarjan = 1 pass, Kosaraju = 2 passes + reversed graph), but this comparison **does not appear in the paper**. The paper mentions only Tarjan (1972) by name. When you present this, make clear it is supplementary knowledge you added, not something from Vincent-Lamarre et al.

Action Items for Next Week

1. Re-read Blondin Massé et al. (2008), Algorithm 2. Convince yourself that out-degree is correct by tracing through the *fast/speed/moving* example by hand.
2. Look at `tests/test_kernel_outdegree.py` in this repository. The test `test_cycle_plus_tail_buggy_w` explicitly shows the divergence between the correct and buggy algorithms.
3. Try implementing `compute_kernel` yourself before looking at `src/rs_dic_llm/metrics/kernel_core.py`